Computer Vision Techniques — Answer Sheet

Unit 1: Image Representation & Basics

> **Key Focus Topics:** Filtering Operations | Histogram Equalization

---

PART A — SHORT QUESTIONS (1–14)

Image Representation & Basics

Q1. How is a grayscale image represented mathematically? Intensity levels for 8-bit?

A grayscale image is represented as a **2D matrix** `f(x, y)` where:

- `x, y` → pixel position (row, column)
- `f(x, y)` → intensity (brightness) value at that pixel
- Values range from **0 (black)** to **255 (white)**

8-bit image → 2^8 = 256 possible intensity levels

Think of the image as a grid of boxes. Each box holds a single number (0–255) that tells how bright that spot is. The more bits you use, the more shades of gray you can store. 8-bit gives 256 shades — enough for smooth, natural-looking images. Higher bit-depth (like 16-bit) gives 65,536 levels, used in medical/scientific imaging.

> **Memory tip:** 8 bits → 256 levels → 0 is pure black, 255 is pure white.

Q2. Compare Grayscale vs Color (RGB) representation

Feature	Grayscale	RGB Color
Values per pixel	**1**	**3** (R, G, B)
Storage	Less (1× memory)	More (3× memory)
Information	Brightness only	Color + Brightness
Use case	Medical, edge detection	Natural images, UI

In grayscale, each pixel has **one number** — how bright it is. In RGB, each pixel has **three numbers** — how much Red, Green, and Blue light it contains. Mixing R, G, B at different levels gives millions of colors. For many CV tasks (edge detection, thresholding), grayscale is preferred because it's simpler and faster to process.

> **Memory tip:** RGB = 3 channels = 3× the data of grayscale. Grayscale = just brightness, no color info.

Q3. Explain: `g(x, y) = T[f(x, y)]`

This is the **general image transformation equation**:

- `f(x, y)` → **input image** — original pixel value at position (x, y)
- `T` → **transformation operator** — any operation applied to each pixel (brighten, blur, negate, etc.)
- `g(x, y)` → **output image** — result after applying T to f

Think of `T` as a rule applied to every pixel of the input to produce the output. It can be a simple point operation (one pixel at a time) or a neighborhood operation (uses surrounding pixels). The equation captures every image processing operation in one clean notation.

Example: Image negation → `g(x, y) = 255 - f(x, y)` A pixel at value 50 → output becomes 205 (dark becomes bright)

> **Memory tip:** f = input, T = what you do to it, g = output. Like: raw dough (f) → oven (T) → baked bread (g).

Point Processing & Intensity Transformations

Q4. Image negation for pixels [20, 50, 100, 200] with L = 256

Formula: `s = (L - 1) - r = 255 - r`

Input (r)	Calculation	Output (s)
20	255 - 20	**235**
50	255 - 50	**205**
100	255 - 100	**155**
200	255 - 200	**55**

Image negation flips brightness — dark pixels become light and light pixels become dark. This is exactly like a photo negative. It is useful in medical imaging to reveal features that are hard to see in the original (e.g., bright tumors on dark X-ray backgrounds become dark on white background — easier to spot). The transformation is linear and reversible.

> **Memory tip:** Negation = mirror around 127.5. High values go low, low values go high. Apply twice → get original back.

Q5. Log transformation and its use in enhancing dark images

Log transformation is a non-linear image enhancement technique that boosts the visibility of details in dark images by mapping a narrow range of low-intensity input values into a wider range of higher-intensity output value. It operates by applying a logarithmic function to each pixel, which effectively expands dark pixels while compressing brighter pixels

Formula: `s = c × log(1 + r)`

- `r` = input pixel intensity, `s` = output, `c` = scaling constant
- **Effect:** Compresses the bright end, **expands the dark end**
- Dark pixels (small r) → mapped to proportionally larger s → become visible
- Very bright pixels are compressed — they don't dominate the image

Application:

- Enhancing dark regions in X-rays and medical scans
- Displaying the Fourier spectrum (which has a huge dynamic range)
- Making shadow detail visible in outdoor photos

The `+1` inside the log ensures that when r = 0, log(1) = 0 — so black remains black. The constant `c` is chosen to scale the output back to 0–255.

> **Memory tip:** Log = "magnifying glass for dark pixels." Small values grow big, big values barely change. Great for dark X-rays.

Q6. Gamma transformation `s = c × r^γ` — effect of γ

The **Gamma Transformation** (or Power-Law Transformation) is a fundamental technique in digital image processing used to adjust the brightness and contrast of an image, specifically to map pixel intensities in a non-linear way.

γ value	Curve shape	Effect on image	Real-world use
γ < 1	Concave up	**Brightens** (darks → lighter)	Underexposed / dark images
γ = 1	Straight line	No change	Baseline / identity
γ > 1	Concave down	**Darkens** (brights → darker)	Overexposed / washed-out images

Gamma correction is widely used in monitors, cameras, and televisions. When you adjust screen brightness or contrast, it often applies a gamma curve. Values below 1 lift the shadows while values above 1 push the highlights down. This is also called **power-law transformation** and is the basis of display calibration.

> **Memory tip:** γ < 1 = Boost (brighter). γ > 1 = Suppress (darker). Think of γ like a volume knob — below 1 turns up the darks, above 1 brings down the brights.

Histogram & Enhancement ⭐ (Important Topic)

Q7. Define histogram equalization. Why useful for low-contrast images?

Histogram equalization is a technique that **redistributes pixel intensity values** so that all gray levels are used as evenly as possible across the full range (0–255).

Why it works on low-contrast images:

- Low contrast images → pixel values packed in a narrow range (e.g., all between 100–150)
- The histogram looks like a tall narrow spike in the center

- After equalization → values spread across full range (0–255)
- The histogram looks flat (uniform) → more visible contrast → hidden details emerge

Concept of steps:

1. Compute the histogram (count of each intensity)
2. Compute the CDF (cumulative sum of probabilities)
3. Use CDF as a mapping function to remap old intensities to new ones

It is like taking a histogram that looks like a narrow tall spike and stretching it flat across the full width. Common applications: medical imaging, satellite images, foggy photographs.

> **Memory tip:** Equalization = "flatten the histogram." Squished = low contrast; spread out = high contrast. CDF is the bridge between old and new values.

Q8. Calculate PDF for the given pixel frequency

Total pixels = 10 + 20 + 30 + 40 = **100**

PDF (Probability Density Function) = Frequency ÷ Total pixels

Gray Level	Frequency	PDF
0	10	**0.10**
1	20	**0.20**
2	30	**0.30**
3	40	**0.40**

PDF tells you: *"What fraction of pixels have this gray level?"* It is a normalized histogram. The sum of all PDFs must equal 1.0, which confirms all pixels are accounted for. PDF is the first step in histogram equalization — you use it to compute the CDF, which then gives the new gray level mapping.

> **Memory tip:** PDF = count / total. All PDF values must sum to 1. Think of it as probability — how likely is a pixel to have this level?

Filtering & Edge Detection ⭐ (Important Topic)

Q9. Smoothing filters vs Sharpening filters

Smoothing filters reduce noise and blur the image:

- Work by **averaging or weighting** neighboring pixels
- Reduce high-frequency components (noise, fine detail)
- Useful as preprocessing: clean the image before edge detection or thresholding
- Side effect: edges also get blurred slightly

Sharpening filters enhance edges and fine detail:

- Work by **emphasizing differences** between neighboring pixels
- Detect rapid intensity changes (= edges)
- Contain negative values in the kernel — they subtract the smooth background
- Can amplify noise if applied to an unclean image

Feature	Smoothing	Sharpening
Purpose	Noise removal, blur	Edge/detail enhancement
Effect on image	Softer, blurred	Crisper, more defined
Kernel values	All positive, sum = 1	Contains negatives
Effect on noise	Reduces it	Can amplify it
Example filters	Mean, Gaussian, Median	Laplacian, Sobel, Prewitt
Frequency response	Removes high freq (noise)	Enhances high freq (edges)

Common practice: **smooth first, then sharpen** — this way you remove noise before trying to detect edges.

> **Memory tip:** Smooth = blend in (neighbors agree). Sharpen = stand out (highlight disagreements between neighbors). Smooth before you sharpen.

Q10. Difference between Sobel and Prewitt operators

Both are **gradient-based edge detection** operators. They compute the rate of intensity change in X (horizontal) and Y (vertical) directions to find edges.

Feature	Prewitt	Sobel
Kernel weights	All 1s	Center row/col weighted (2s)
Noise sensitivity	Moderate	**Lower (more robust)**
Smoothing effect	Less	**More (built-in smoothing)**
Complexity	Simpler	Slightly more complex
Preferred for	Simple images	Real-world noisy images

Sobel X kernel (detects vertical edges):

```
-1   0  +1
-2   0  +2
-1   0  +1
```

Prewitt X kernel (detects vertical edges):

```
-1   0  +1
-1   0  +1
```

```
 -1   0  +1
```

The key difference: Sobel's **center row has weight 2** — this acts like a built-in Gaussian smoothing pass, making it more tolerant to noise. Prewitt treats all rows equally (weight 1), so it is simpler but noisier. Both detect edges in X and Y; combine results as `|G| = √(Gx² + Gy²)` for full edge magnitude. In practice, Sobel is always preferred for real images.

> **Memory tip:** Sobel = Smarter (built-in smoothing, weight 2 in center). Prewitt = Plain (equal weights). For noisy real-world images, always choose Sobel.

Thresholding

Q11. Global vs Adaptive thresholding

Global thresholding: One single threshold value `T` applied to the whole image.

- If `pixel > T` → white (foreground); if `pixel ≤ T` → black (background)
- Works well with uniform, well-lit images
- Fails when lighting varies across the image (shadows, uneven illumination)
- Simple and fast, but fragile in real-world conditions

Adaptive thresholding: Computes a **different threshold for each local region** of the image.

- Image divided into small windows/blocks; each gets its own local T
- Local T based on mean or Gaussian-weighted mean of that neighborhood
- Works well even with shadows, faded documents, uneven backgrounds
- Slightly more computation but much more robust

Feature	Global	Adaptive
Threshold value	One for whole image	Different per region
Lighting condition	Uniform	Uneven / shadows
Computation	Simple, fast	More compute needed
Example use	Clean scanned docs	Real photos, receipts, outdoor scenes
Failure case	Shadows cause misclassification	Rarely fails

> **Memory tip:** Global = one rule for all. Adaptive = different rules for different neighborhoods. If there are shadows → always use adaptive.

Q12. Why Otsu thresholding suits bimodal histograms?

A **bimodal histogram** has two distinct peaks — one for the background (dark) and one for the foreground (bright). A classic example: a white paper with black text, or a bright metal surface with dark defects.

Otsu's method automatically finds the optimal threshold between the two peaks:

1. Tries every possible threshold value (0–255)
2. For each threshold T, splits pixels into two classes
3. Calculates **within-class variance** (how spread each group is internally)
4. Picks the T that **minimizes within-class variance** (= maximizes separation between classes)
5. No manual input needed — fully automatic

Why bimodal works perfectly: When two clear groups exist, the optimal split is at the valley between the two peaks. Otsu finds this valley automatically. If the histogram is unimodal (one hump), Otsu may not find a meaningful split.

> **Memory tip:** Otsu = "auto-threshold for two groups." Bimodal = two humps = easy valley to split in. Otsu finds the deepest valley automatically.

Interest Points

Q13. What are keypoints? Why important?

Keypoints (also called interest points) are **distinctive, stable locations** in an image — typically corners, blobs, or areas with high texture variation that stand out from their surroundings.

Properties of good keypoints:

- **Repeatable:** Found at the same location even after rotation, scaling, or cropping
- **Distinctive:** Unique enough to match between different images of the same scene
- **Localized:** Precise in position (and ideally scale)

Why important in computer vision:

- **Object recognition:** Match keypoints in a query image to a database image
- **Image stitching:** Align overlapping photos to create panoramas
- **3D reconstruction:** Match points across frames to compute depth (stereo vision)
- **Video tracking:** Follow objects frame-to-frame using consistent keypoints
- **Augmented reality:** Anchor virtual objects to real-world keypoints stably

> **Memory tip:** Keypoints = landmark features. Like navigating a city by famous landmarks (not every street). Stable, unique, easy to find again.

Q14. Main stages of SIFT algorithm

SIFT (Scale-Invariant Feature Transform) has 5 main stages:

1. **Scale-space construction** — Apply Gaussian blur at multiple scales; compute Difference of Gaussians (DoG) to find blob-like structures at all scales
2. **Keypoint detection** — Find local extrema (max/min) in DoG across scale and spatial position
3. **Keypoint localization** — Remove weak, low-contrast, or edge-only keypoints to keep only stable ones
4. **Orientation assignment** — Compute dominant gradient direction at each keypoint → makes descriptor rotation-invariant

5. **Descriptor generation** — Build a 128-dimensional feature vector from gradient histograms in a 4×4 grid around the keypoint

Each stage refines and filters keypoints, so only the most stable and distinctive points survive as final SIFT features. The 128-D vector is like a unique fingerprint for each keypoint.

> **Memory tip:** SIFT stages = **S**cale-space → **I**dentify extrema → **F**ilter weak → **T**urn to descriptor. The acronym teaches itself!

PART B — LONG / SCENARIO / NUMERICAL (15–20 Marks Each)

Scenario 1: Industrial Inspection — Defect Detection on Metal Surfaces (15 Marks)

> *A factory uses CV to detect defects on metal surfaces. Images contain noise and uneven illumination.*

Introduction

In industrial quality control, automated visual inspection systems replace manual checking. Metal surfaces may have defects like cracks, dents, or foreign particles. Raw images often suffer from **salt-and-pepper noise** (from camera sensors) and **uneven illumination** (from lighting rigs). A systematic image processing pipeline must address both issues before attempting detection.

---

a) Preprocessing Steps (in order)

Step 1: Grayscale Conversion

- Metal surface defects are structural — color adds no useful information here
- Convert RGB → Grayscale to reduce data and simplify processing
- Formula: `f_gray = 0.299R + 0.587G + 0.114B`

Step 2: Noise Removal

- Camera and sensor introduce random noise (bright/dark speckles)
- Apply **Median filter** — best for salt-and-pepper noise, preserves edges
- Alternatively, apply **Gaussian filter** for continuous Gaussian-type noise

Step 3: Illumination Normalization

- Uneven lighting creates brightness gradients that look like defects
- Apply **histogram equalization** or **CLAHE** (Contrast Limited Adaptive Histogram Equalization)
- CLAHE is preferred: it equalizes locally so bright and dark regions are both enhanced without over-amplifying noise

Step 4: Contrast Enhancement

- Apply **log transformation** to make subtle surface defects (low-contrast scratches) more visible
- This ensures defects are distinguishable from the normal background

Step 5: Morphological Operations (optional)

- Dilation/erosion can fill small gaps in detected defect regions
- Removes tiny noise blobs that might pass thresholding

---

b) Suitable Filters

Median Filter — Primary choice for noise removal:

- Replaces each pixel with the **median** of its 3×3 or 5×5 neighborhood
- Extremely effective for **salt-and-pepper noise** (random white/black pixels from sensors)
- Key advantage: **Preserves edges** unlike mean filter which blurs them
- Since defect boundaries are critical, preserving them is essential for accurate detection

```
Noisy neighborhood:        After median:
[ 5  200  10 ]
[12   15  14 ]   →   Center pixel = median of {5,200,10,12,15,14,11,13,9}
= 13
[11   13   9 ]
```

Gaussian Filter — Secondary smoothing:

- Applies weighted average using a bell-shaped kernel (center pixels weighted more)
- Effective for **Gaussian/electronic noise**
- Smooths image while keeping strong edges mostly intact
- Often applied before Sobel edge detection to reduce false edge responses

Mean Filter — Avoid for this case:

- Averages all neighbors equally → blurs edges and defect boundaries
- Not suitable when precise defect localization and edge sharpness are needed

Filter	Noise Type	Edge Preservation	Recommended?
Median	Salt & Pepper	Yes (best)	✅ First choice
Gaussian	General/Electronic	Partial	✅ Secondary
Mean	General	No (blurs)	❌ Avoid

---

c) Recommended Thresholding: Otsu's Thresholding

Why Otsu:

- Metal surface images naturally produce **bimodal histograms** — one peak for normal surface, another for defects
- Otsu automatically finds the optimal threshold in the valley between the two peaks
- No manual tuning required — works consistently across batches
- Robust to slight illumination changes after preprocessing

How Otsu works step-by-step:

1. Compute histogram of the preprocessed image
2. For each possible T (0–255): compute weighted within-class variance for the two groups (below T and above T)
3. Select T that **minimizes within-class variance** = best separation of defect vs. normal surface
4. Apply threshold: below T → defect (black), above T → normal surface (white)

Alternative for uneven illumination: Use **Adaptive Otsu** or **Sauvola's method** — compute local thresholds in sliding windows for each region.

Complete pipeline:

```
Capture image → Grayscale → Median filter → Histogram equalization → Otsu
threshold → Defect map
```

Scenario 2: Medical Imaging — Enhance Dark Regions in X-ray (15 Marks)

> *A doctor wants to enhance dark regions in an X-ray image.*

Introduction

X-ray images often have regions of interest (soft tissue, early-stage tumors, fluid accumulation) that appear very dark and are hard to distinguish from the background. The goal is to selectively brighten dark areas without overexposing bright bone regions. This requires careful non-linear intensity transformation.

---

a) Best Transformation: Log Transformation

Log transformation is the **best choice** for enhancing dark regions because:

- It **non-linearly maps** input intensities: small (dark) values expand a lot, large (bright) values compress slightly
- Reveals hidden detail in shadows without blowing out bright areas (bones)
- Preserves the relative order of intensities — no structural distortion

---

b) Formula and Step-by-step Explanation

Formula: `s = c × log(1 + r)`

- `r` = original pixel intensity (0–255)

- `s` = transformed output pixel
- `c` = scaling constant (usually `255 / log(256) ≈ 45.9`) to keep output in 0–255

Pixel-by-pixel behavior (c = 1, natural log):

Input r	1 + r	log(1 + r)	Behavior
0 (black)	1	0.00	Black stays black
10 (very dark)	11	2.40	Mapped up significantly
50 (dark gray)	51	3.93	Noticeable brightening
100 (mid gray)	101	4.61	Moderate increase
200 (light gray)	201	5.30	Slight increase only
255 (white)	256	5.55	Compressed — still bright

Why `1 + r`? Avoids log(0) = undefined. When r = 0 → log(1) = 0 → black stays black.

Visual effect in X-ray:

- Dark soft tissue regions → brightened significantly → visible to radiologist
- Bright bone regions → slight compression → still clearly distinguishable
- Overall: more diagnostic information visible without distorting the image structure

---

c) Log Transformation vs Histogram Equalization

Aspect	Log Transformation	Histogram Equalization
Enhancement type	Targets **dark regions specifically**	Global contrast enhancement
Predictability	High — same formula, same result	Depends on image histogram
Effect on bright areas	Slight compression	Can over-brighten
Effect on dark areas	Strong, controlled brightening	Depends on pixel distribution
Risk of artifacts	Very low	Can create false patterns
Formula	`s = c × log(1 + r)`	CDF-based remapping
Best for	X-rays, dark region detail	Low-contrast foggy images
Medical safety	✅ Preferred	Use cautiously

Conclusion for X-ray:

- Log transformation is safer — smooth, predictable, mathematically controlled
- Histogram equalization may over-enhance uniform dark regions creating false texture
- Radiologists rely on consistent enhancement → log transformation is the standard choice

---

Scenario 3: Autonomous Vehicle — Edge Detection Under Varying Lighting (15 Marks)

> *An autonomous vehicle needs reliable edge detection under varying lighting conditions.*

Introduction

Edge detection is critical for autonomous vehicles to identify lane markings, pedestrians, road boundaries, and obstacles. Under varying lighting (sunlight, shadows, night, headlights), the edge detector must be robust to noise and false edges caused by illumination changes. Choosing the right operator is crucial for safety.

---

a) Full Comparison: Sobel, Prewitt, Roberts

Roberts Cross Operator (2×2):

- Simplest and oldest edge detector
- Detects edges at 45° diagonals using 2×2 kernels
- Very fast but highly sensitive to noise — any random pixel fluctuation triggers a false edge
- Not suitable for real-world outdoor conditions

```
Gx:          Gy:
+1   0       0   +1
 0  -1      -1    0
```

Prewitt Operator (3×3):

- Uses 3×3 kernel — considers more neighbors → better noise tolerance than Roberts
- Equal weights in all rows → no built-in smoothing
- Detects horizontal and vertical edges
- Moderate performance, acceptable for controlled indoor settings

```
Gx:            Gy:
-1  0  +1      -1  -1  -1
-1  0  +1       0   0   0
-1  0  +1      +1  +1  +1
```

Sobel Operator (3×3):

- Also 3×3, but **center row/column has weight 2** — this creates built-in Gaussian smoothing
- Best noise suppression of the three
- Industry standard for real-world edge detection
- Detects edges in X and Y; combine: `|G| = √(Gx² + Gy²)`

```
Gx:              Gy:
-1  0  +1        -1  -2  -1
-2  0  +2         0   0   0
-1  0  +1        +1  +2  +1
```

Full comparison:

Feature	Roberts	Prewitt	Sobel
Kernel size	2×2	3×3	3×3
Noise sensitivity	High	Medium	**Low (best)**
Built-in smoothing	None	None	Yes (weight-2 center)
Edge direction	45° diagonal	H + V	H + V
Speed	Fastest	Medium	Medium
Accuracy on noisy images	Poor	Moderate	**Best**
Best for	Clean lab images	General	**Real-world, outdoor**

---

b) Most Robust: Sobel

Sobel is most robust for autonomous vehicles because:

1. **Built-in smoothing:** Weight-2 center effectively applies 1D Gaussian before differencing → suppresses noise from sunlight flicker and shadow edges
2. **3×3 neighborhood:** Uses 9 pixels instead of 4 (Roberts) → more stable, spatially consistent response
3. **H + V detection:** Detects edges at any angle → lane lines, pedestrian outlines, curbs are all captured
4. **Real-world proven:** Used in Canny edge detector, OpenCV default, automotive vision systems

Recommended pipeline for AV edge detection:

```
Raw frame → Grayscale → Gaussian pre-blur → Sobel (Gx + Gy) → Gradient
magnitude → Threshold → Edge map
```

> For best results, use **Canny detector** which builds on Sobel internally and adds non-maximum suppression + hysteresis thresholding — giving the cleanest, thinnest edges possible.

⭐ Numerical 4: Image Negation (Matrix)

Formula: `s = 255 - r`

Given input matrix:

```
[ 50   120 ]
[ 200  255 ]
```

Step-by-step:

- 255 - 50 = **205** (dark gray → light gray)
- 255 - 120 = **135** (medium gray → flipped medium gray)
- 255 - 200 = **55** (light gray → dark gray)
- 255 - 255 = **0** (white → black)

Output matrix:

```
[ 205  135 ]
[  55    0 ]
```

Interpretation: The output is the photographic negative. Bright areas become dark and dark areas become bright. Useful in medical imaging where features may be more visible after inversion (e.g., bright white tumor on dark background vs. dark tumor on white background).

⭐ Numerical 5: Log Transformation

Formula: `s = log(1 + r)` (c = 1, natural log)

r	Step 1: 1 + r	Step 2: ln(1 + r)	Output s
0	1	0.000	**0.00**
10	11	ln(11)	**2.40**
50	51	ln(51)	**3.93**
200	201	ln(201)	**5.30**

Key observation: Input range spans 0–200 (a range of 200), but output only spans 0–5.3 (range of just 5.3).

- Dark values (0, 10) → expanded and brightened significantly
- High value (200) → compressed to just 5.30
- This is the "stretch darks, compress brights" effect of log transformation

---

⭐ Numerical 6: Gamma Correction

Formula: `s = r^0.5` (γ = 0.5, normalized inputs 0–1)

r (input)	Calculation	s (output)	Change
0.1	$\sqrt{0.1}$	**0.316**	+0.216 (large boost for dark pixels)

r (input)	Calculation	s (output)	Change
0.4	$\sqrt{0.4}$	**0.632**	+0.232 (moderate boost)
0.8	$\sqrt{0.8}$	**0.894**	+0.094 (small boost for bright pixels)

Interpretation:

- $\gamma = 0.5 < 1$ → all output values are **larger than inputs** → image gets **brighter overall**
- Dark pixels receive the biggest proportional boost → shadow detail becomes visible
- Bright pixels receive only a small boost → highlights stay controlled
- Ideal for correcting underexposed images

⭐ Numerical 7: Histogram Equalization — Full Worked Example (KEY TOPIC)

Given: Total pixels = 100, L = 4 (gray levels 0–3)

Step 1: Compute PDF (Probability Density Function)

```
PDF(level) = count / total pixels
```

Level	Count	PDF
0	10	**0.10**
1	20	**0.20**
2	30	**0.30**
3	40	**0.40**

PDF sum = 0.10 + 0.20 + 0.30 + 0.40 = **1.00** ✅

Step 2: Compute CDF (Cumulative Distribution Function)

```
CDF(level) = sum of all PDFs up to and including this level
```

Level	PDF	CDF
0	0.10	**0.10**
1	0.20	**0.30** (= 0.10 + 0.20)
2	0.30	**0.60** (= 0.30 + 0.30)
3	0.40	**1.00** (= 0.60 + 0.40)

CDF always ends at 1.0 — confirms all pixels accounted for.

Step 3: New Gray Level Mapping

`New level = round(CDF × (L − 1)) = round(CDF × 3)`

Old Level	CDF	CDF × 3	Rounded	New Level
0	0.10	0.30	0	**0**
1	0.30	0.90	1	**1**
2	0.60	1.80	2	**2**
3	1.00	3.00	3	**3**

Interpretation

In this specific example the mapping is 1:1 because the distribution is already reasonably spread. In real low-contrast images, you would see many old levels collapsing to the same new level at one end (compression of peaks) while underrepresented levels are spread out across the range.

What equalization achieves in practice:

- Pixels crammed in the 100–150 midtone range get mapped to span 0–255
- Histogram that looked like a narrow spike becomes flat-ish
- Visual result: much higher contrast, previously invisible details become clear

Where it is used:

- Medical X-rays — reveal soft tissue detail
- Satellite images — improve terrain visibility
- Foggy/hazy photos — restore contrast lost to atmospheric scattering

---

⭐ Numerical 8: Convolution with Mean Filter — KEY TOPIC (Filtering Operations)

Mean filter kernel (3×3):

```
    [ 1  1  1 ]
1/9 [ 1  1  1 ]
    [ 1  1  1 ]
```

Input image patch:

```
[ 10  20  30 ]
[ 40  50  60 ]
[ 70  80  90 ]
```

Step-by-step Convolution

Step 1: Align kernel over the center pixel Each kernel position multiplies with the corresponding image pixel. Since all kernel values = 1/9, we just need the sum of all image values.

Step 2: Multiply and accumulate

```
(1/9 × 10) + (1/9 × 20) + (1/9 × 30)
+ (1/9 × 40) + (1/9 × 50) + (1/9 × 60)
+ (1/9 × 70) + (1/9 × 80) + (1/9 × 90)
= (1/9) × (10+20+30+40+50+60+70+80+90)
```

Step 3: Sum all values `10 + 20 + 30 + 40 + 50 + 60 + 70 + 80 + 90 = 450`

Step 4: Divide by 9 `450 / 9 = 50`

Output center pixel = 50

What the Mean Filter Does

The mean filter replaces each pixel with the **average of its 3×3 neighborhood**. In this symmetric example, the average happens to equal the center pixel (50). But in a noisy image, the real power shows:

Noise removal example:

```
[ 50  52  48 ]
[ 51  200  49 ]   ← center pixel = 200 (noise spike)
[ 53  47  51 ]
```

Sum = 50+52+48+51+200+49+53+47+51 = 601 → Mean = 601/9 ≈ 67 The noise spike (200) is pulled down to ~67 — much closer to the surrounding values.

Mean Filter vs Gaussian Filter vs Median Filter

Feature	Mean Filter	Gaussian Filter	Median Filter
Kernel weights	Equal (1/9 each)	Gaussian bell (center weighted)	No kernel — uses sorting
Noise removal	Good for Gaussian noise	Better for Gaussian noise	Best for salt & pepper
Edge preservation	Poor (blurs edges)	Moderate	**Best (preserves edges)**
Computation	Simplest	Moderate	More (requires sorting)
When to use	Quick smoothing	Before edge detection	When edges must be kept

---

Effect of Filter Size

Kernel Size	Effect
3×3	Light smoothing, minimal blur
5×5	More smoothing, noticeable blur
7×7	Heavy blur, strong noise removal

Larger kernel = more neighbors averaged = smoother result = more blur of edges. Choose kernel size based on trade-off between noise removal and edge preservation.

⭐ Scenario 9: Thresholding with Shadows — Adaptive Thresholding (15 Marks)

> *An image contains shadows causing uneven brightness.*

Introduction

Thresholding converts a grayscale image into a binary (black/white) image by classifying pixels as either foreground or background. It is one of the simplest yet most powerful segmentation tools. However, its effectiveness depends critically on the choice of method when image conditions are non-ideal.

---

a) Why Global Thresholding Fails

Global thresholding applies **one fixed threshold T** to the entire image:

- Pixels above T → white (foreground)
- Pixels below T → black (background)

Failure scenario with shadows:

- Consider a white paper with text photographed under uneven light
- Bright area: paper pixel = 190, text pixel = 80 → T = 128 correctly separates (190 > 128 = paper, 80 < 128 = text) ✅
- Shadow area: paper pixel = 100, text pixel = 40 → T = 128 **misclassifies both as text** ❌
- The paper in the shadow region (100) is below the threshold → wrongly labeled as foreground

Root cause: One global T cannot simultaneously be correct for a bright illuminated region AND a dark shadowed region. The histogram has multiple overlapping clusters rather than two clean peaks.

Visual result: The binary image shows broken, fragmented foreground regions where shadows were present — unusable for OCR, counting, or shape analysis.

---

b) Better Method: Adaptive (Local) Thresholding

Instead of one global T, compute a **local threshold for every region** of the image independently.

---

c) Working Principle of Adaptive Thresholding

Step 1: Choose a window size

- Typical sizes: 15×15, 31×31, or 51×51 pixels
- Window should be large enough to contain meaningful local statistics
- Too small → noisy threshold; too large → misses local variation

Step 2: Slide the window across the image

- For each pixel position (x, y), consider the NxN neighborhood centered on it
- This is a sliding window approach

Step 3: Compute local threshold

Method A — Mean Adaptive: `T(x, y) = mean of neighborhood pixels - C`

- C is a small constant (typically 5–15) to account for local bias
- Simple and fast

Method B — Gaussian Adaptive: `T(x, y) = Gaussian-weighted mean of neighborhood - C`

- Pixels closer to center are weighted more → more representative of local background
- More accurate, slightly more computation

Step 4: Classify each pixel

- Compare the pixel value to its **own local threshold**
- `pixel(x, y) > T(x, y)` → white (background or foreground depending on convention)
- `pixel(x, y) ≤ T(x, y)` → black

Step 5: Result

- Shadowed region: local T is low (dark neighborhood) → shadow pixels correctly classified
- Bright region: local T is high → bright pixels correctly classified
- Each region gets a threshold appropriate for its own lighting condition

---

Full Comparison Table

Feature	Global Thresholding	Adaptive Thresholding
Threshold value	One for full image	Different per local window
Works for uniform lighting	✅ Yes	✅ Yes
Works for shadows/uneven light	❌ No	✅ Yes
Computation	Simple, fast	More computation
Parameters	Just T	Window size, C constant
Real-world robustness	Low	High
Example failures	Shadows, gradients	Rarely fails

Feature	Global Thresholding	Adaptive Thresholding
Example use cases	Uniform lab scans	Documents, outdoor images

---

⭐ Long Q10: Complete SIFT Pipeline — Scale and Rotation Invariance (20 Marks)

> *Explain SIFT pipeline and discuss why it is scale and rotation invariant.*

Introduction

SIFT (Scale-Invariant Feature Transform), proposed by David Lowe in 2004, is one of the most influential local feature detection and description algorithms in computer vision. It detects distinctive keypoints in images and generates descriptors that remain stable across changes in **scale, rotation, and moderate illumination variation**. This makes it ideal for object recognition, image stitching, 3D reconstruction, and visual SLAM.

Stage 1: Scale-Space Construction

Goal: Detect features that are stable at multiple scales (image sizes/zooms).

Process:

1. Start with the original image
2. Progressively blur using Gaussian filter with increasing σ: `G(x, y, σ)` at σ = σ, kσ, k²σ, ...
3. This creates an **image pyramid** — each "octave" contains several blurred versions
4. Compute **Difference of Gaussians (DoG):** `DoG(x, y, σ) = G(x, y, kσ) − G(x, y, σ)`
5. DoG approximates the Laplacian of Gaussian — a blob and edge detector at each scale

```
Original → Blur₁ → Blur₂ → Blur₃ → Blur₄ → Blur₅
                ↓         ↓         ↓         ↓
              DoG₁      DoG₂      DoG₃      DoG₄
```

Why this matters: An object at different scales (close vs. far) will appear as different feature sizes. Scale-space ensures the same object feature is detected regardless of distance/zoom.

Stage 2: Keypoint Detection

Goal: Find locations and scales where image structure is distinctive.

Process:

1. In the DoG stack, find pixels that are **local extrema** (local max or min)
2. A pixel is an extremum if it is greater (or less) than ALL 26 neighbors:
   - 8 neighbors in the same scale DoG layer
   - 9 neighbors in the DoG layer above (coarser scale)
   - 9 neighbors in the DoG layer below (finer scale)

These 26-neighbor comparisons ensure the keypoint is a true local extremum in both space and scale.

Result: A large set of candidate keypoints at various (x, y, scale) positions.

Stage 3: Keypoint Localization (Refinement and Filtering)

Goal: Remove unstable, low-quality keypoints. Keep only reliable ones.

Sub-step A — Sub-pixel localization:

- Fit a 3D quadratic to refine keypoint position to sub-pixel accuracy
- Improves localization precision

Sub-step B — Low contrast rejection:

- Evaluate DoG magnitude at the keypoint position
- If DoG < threshold (typically 0.03) → discard (unstable keypoint, sensitive to noise)

Sub-step C — Edge keypoint rejection using Hessian:

- Edges have high gradient in one direction only → poor distinctive keypoints
- Compute the 2×2 Hessian matrix at the candidate
- If ratio of eigenvalues (curvatures) exceeds threshold → it is an edge → reject
- Only corners/blobs (high curvature in both directions) pass this filter

Result: A refined, clean set of stable, high-quality keypoints.

Stage 4: Orientation Assignment

Goal: Assign a reference orientation to each keypoint → enables rotation invariance.

Process:

1. For each keypoint, define a local neighborhood scaled to the keypoint's scale
2. Compute gradient magnitude and direction at every pixel in the neighborhood
3. Weight contributions by a Gaussian centered on the keypoint (nearby pixels matter more)
4. Build a **36-bin orientation histogram** (each bin = 10°, covering 360°)
5. Find the peak bin → that angle becomes the **reference orientation** of the keypoint
6. If another peak ≥ 80% of the dominant peak → create an additional keypoint copy with that orientation (handles symmetry)

Why this gives rotation invariance: All subsequent computations are performed relative to this dominant orientation. When the image rotates, the orientation changes with it — but the descriptor, computed relative to that orientation, stays the same.

Stage 5: Descriptor Generation

Goal: Create a compact, distinctive, rotation-normalized representation.

Process:

1. Rotate the coordinate frame to align with the keypoint's dominant orientation
2. Take a **16×16 pixel patch** around the keypoint (at the keypoint's scale)
3. Divide the patch into a **4×4 grid of cells** (each cell = 4×4 pixels)
4. For each of the 16 cells, compute an **8-bin gradient orientation histogram**
5. Concatenate all 16 histograms: **4 × 4 × 8 = 128 values**
6. Normalize the 128-D vector (divide by its L2 norm) → reduces illumination sensitivity
7. Clamp values at 0.2 and re-normalize → reduces the influence of nonlinear illumination

Result: A **128-dimensional SIFT descriptor** — the "fingerprint" of that keypoint. Two SIFT descriptors can be compared using Euclidean distance to find matching keypoints across different images.

Why SIFT is Invariant — Summary

Property	Mechanism	Practical benefit
Scale invariant	Keypoints detected in DoG scale-space. Each keypoint has its own scale. Descriptor computed at that scale.	Same keypoint detected whether object is near or far
Rotation invariant	Dominant orientation assigned to each keypoint. Descriptor aligned to that orientation.	Same descriptor even if image is rotated
Partially illumination invariant	Descriptor uses gradient directions (not absolute intensities). L2 normalization + clamping applied.	Tolerant to moderate brightness and contrast changes
Partially affine invariant	Scale + orientation handling covers mild perspective distortion	Works for small viewpoint changes

---

SIFT Matching Application

Once descriptors are generated for two images, matching proceeds:

1. For each keypoint in image A, find the closest descriptor in image B (nearest neighbor in 128-D space)
2. Apply **ratio test** (Lowe's ratio test): match is accepted only if nearest neighbor distance / second nearest < 0.75
3. This eliminates ambiguous matches

Key strength: SIFT generates thousands of stable keypoints and their 128-D descriptors, enabling reliable matching even when:

- Images are taken from different distances (scale change)
- Camera is rotated
- Partial occlusion occurs
- Moderate lighting differences exist

> **Summary sentence:** SIFT detects the same keypoints and generates the same descriptors regardless of whether the object appears bigger, smaller, or rotated — making it one of the most robust feature descriptors in classical computer vision.

End of Answer Sheet — Unit 1 Key Topics: Filtering Operations (Mean, Gaussian, Median, Sobel) | Histogram Equalization